

LiftZero — Phased Build Plan

Sep 29, 2026 · @Rishiikesh

Overview

LiftZero upgrades the existing elevator fleet coordinator in 7 phases over 7 days. The result is a learned Transformer brain (imitation learning, then PPO) with MCTS look-ahead, running inside a message-driven multi-agent system and shown in a React control-room UI.

Final outcome: five strategies compared on identical seeds, with a live demo of the AI beating the classical system.

Strategy	What decides	Level
NearestCar	Distance rule	Reflex baseline
Classical	A* routing + Contract Net auction + simulated annealing	Current system (the teacher)
LiftZero-IL	Transformer trained to copy the teacher	Supervised learning
LiftZero-RL	Same network after PPO practice	Reinforcement learning
LiftZero-MCTS	Network + tree search over sampled futures	AlphaZero-style

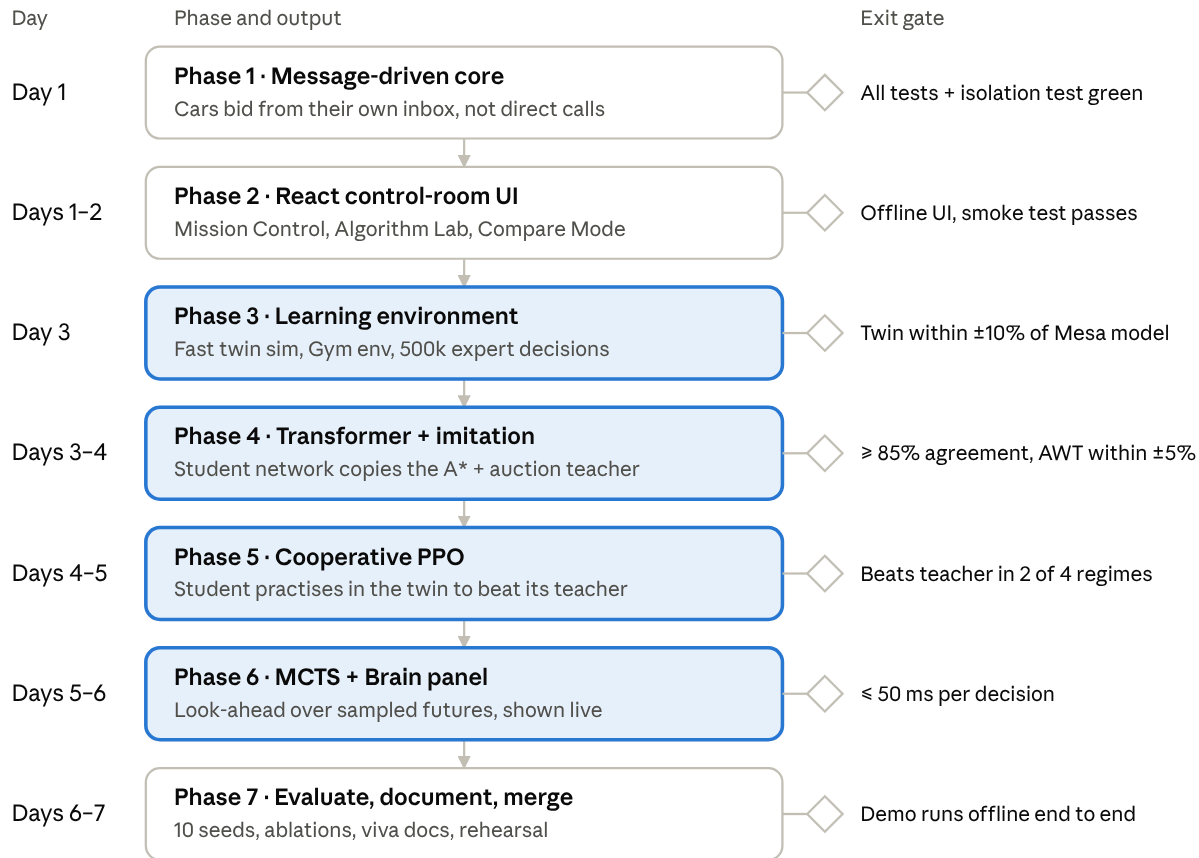
Ground rules

- Today's working version is tagged `v1-stable`. All work happens on a `v2` branch and merges to main only in Phase 7.
- Every phase ends demoable: tests green, `uv run elevator serve` works, committed.
- The demo laptop needs no GPU and no internet: models ship as ONNX files under 5 MB, and the built UI is committed.
- We write the AI algorithms ourselves so the team can explain them. Libraries are for infrastructure only (PyTorch for training, onnxruntime for inference, React for the UI).
- Results are reported honestly, including where LiftZero loses.

Timeline at a glance

Seven phases, each ending in a working demo

Highlighted: the LiftZero learning core (Phases 3–6)



LiftZero roadmap · 7 phases, 7 exit gates

Phases 1–2 make the multi-agent core and the UI solid, Phases 3–6 build the learning core, and Phase 7 proves it and merges.

Phase 1 — Message-driven multi-agent core (Day 1)

Every interaction between agents goes through their own inboxes, and each agent's sense → reason → decide → act cycle is explicit in the code.

Build

- New tick stages: sense → announce → bid → award → commit → act, plus a learn hook for later phases.

- Cars answer the CFP from their own inbox. The dispatcher never calls a car’s methods. Exactly one INFORM per award, and reassignment goes through CANCEL/ACCEPT messages.
- Each car broadcasts its public status (position, direction, load) every tick. This is the shared view LiftZero will read in Phase 4.
- A decision trace for every award: all bids, the cost breakdown and a one-line reason. It feeds the UI now and the training data later.
- Small fixes: `bench` defaults to 900 ticks to match the reports, and `DESIGN.md` §5 matches the code.

Concepts for the viva: FIPA-ACL messages, the Contract Net protocol, agent architecture.

Done when: all 182 existing tests pass, plus new ones: the dispatcher never calls car methods (spy test), one PROPOSE per car per CFP, and exactly one INFORM per award.

Phase 2 — React control-room UI (Days 1-2)

A polished dashboard that works offline and makes the agents' interaction visible at a glance.

Stack: Vite + React + TypeScript, Tailwind + shadcn/ui, Motion for animation, Recharts, React Flow (`@xyflow/react`) for the agent network, Zustand + TanStack Query. API types are generated from FastAPI's OpenAPI schema. The built app goes into `src/elevator_mas/web/dist` , and the old UI stays at `/classic` .

Screens

Screen	What it shows
Mission Control	Animated building (sliding doors, passenger queues, hall lanterns), controls, KPI cards, Auction Theatre, live agent network graph, message swimlanes, rules timeline
Agents	One card per agent: AIMA type, PEAS, live internal state
Algorithm Lab	A* vs UCS vs BFS vs Greedy race; SA vs hill-climbing convergence; minimax tree with alpha-beta pruning (the Brain tab is added in Phase 6)

Screen	What it shows
Experiments	Benchmark runner, plus Compare Mode: two simulations side by side on the same seed
Theory	PEAS, environment classification, architecture, key formulas

Story mode (scripted demo with captions), presentation mode (large fonts) and keyboard shortcuts come with it.

Done when: it works offline at 1366×768 and 1080p, the Vitest tests and one Playwright smoke test pass, and lint and typecheck are clean.

Phase 3 — Learning environment (Day 3)

A fast, faithful training ground for LiftZero, plus a dataset of expert decisions to learn from.

Build

- **Fast twin simulator:** a numpy version of the building for training speed, targeting at least 20× faster than the Mesa model. A fidelity test checks that average wait stays within $\pm 10\%$ of the Mesa model on the same seeds.
- **Gym-style environment:** one decision per new hall call (event-driven, a semi-MDP).
 - Observation: a call token plus one token per car.
 - Action: which car takes the call.
 - Reward: minus the passenger-seconds of waiting and riding added before the next decision, minus a small energy term. It is shared by all cars (a team reward).
- **Features:** car tokens hold position, direction, load, pending stops, door state, ETA and the classical A* marginal cost. The call token holds floor, direction, waiting count, urgency and the traffic pattern.
- **Expert dataset:** about 500k decisions from the Classical strategy, across all scenarios, random seeds and random building sizes (10–40 floors, 2–8 cars). Each decision stores its realised future cost as the value target.

Concepts for the viva: MDP formulation, reward design, generalisation.

Done when: the environment passes determinism and invariant tests, the dataset and its statistics are saved, and the twin fidelity report is within bounds.

Phase 4 — LiftZero network + imitation learning (Days 3–4)

A small Transformer learns to copy the classical expert, and each car uses it to compute its own bid.

Build

- **Model:** a set Transformer with 2–3 layers, 4 attention heads and width 64 (about 0.3M parameters). A per-car score head (the policy) and a value head (expected future fleet cost). It reads the cars as a set, so it works for any number of cars.
- **Decentralised use:** each car scores itself from its own token while attending to the other cars' public status. That score becomes its bid, which still travels as a PROPOSE message, and the auction picks the winner.
- **Training (behaviour cloning):** cross-entropy against the teacher's choice plus a value regression loss. Train and validation sets are split by seed, with early stopping. Training runs on CPU in my cloud workspace.
- **Deployment:** export to ONNX; the laptop runs it with onnxruntime in under 1 ms per bid, with no PyTorch needed.
- New strategy `liftzero_il`, with attention weights exposed to the UI.

Concepts for the viva: supervised learning, attention, generalisation, model deployment.

Done when: at least 85% agreement with the teacher on held-out seeds, average wait within $\pm 5\%$ of Classical, and unseen building sizes still work.

Phase 5 — Cooperative reinforcement learning with PPO (Days 4–5)

The student practises in the simulator until it beats its teacher, with every car learning from one shared team reward.

Build

- **PPO starting from the imitation weights:** the actor is the policy head and the critic is the value head. All cars share parameters and the team reward. GAE ($\lambda = 0.95$), clipping 0.2, an entropy bonus, and a KL guard that keeps it close to the teacher early on.
- **Curriculum:** light traffic $\rightarrow$ two-way $\rightarrow$ peaks $\rightarrow$ faults. Parallel twin environments on 2 CPU cores, training in the background toward 1–2M decisions.
- **Checkpoint choice:** on validation seeds only. The final check runs on the full Mesa model (twin $\rightarrow$ Mesa transfer).

- New strategy `liftzero_r1`, with training curves saved for the UI.

Concepts for the viva: reinforcement learning, policy gradient, actor-critic, credit assignment, cooperative multi-agent learning (centralised training, decentralised execution).

Done when: it beats Classical on average wait in at least 2 of 4 traffic regimes, with 95% confidence over 10 seeds — or there is an honest report of where and why it does not.

Phase 6 — MCTS look-ahead + Brain panel (Days 5–6)

The dispatcher plans ahead over uncertain futures, AlphaZero-style, and the UI shows the model's thinking live.

Build

- **PUCT MCTS for each call:** the root's children are the top-k cars by network prior. Chance nodes are sampled future arrivals from a Bayesian demand model, using the same random draws for every candidate so the comparison is fair. Rollouts run in the twin simulator with the network policy, and leaves are scored by the value head.
- **Time budget:** at most 50 ms per decision (about 64 simulations). If it runs over, it falls back to the plain auction.
- New strategy `liftzero_mcts`.
- **UI "Brain" tab:**
 - A live attention heatmap (which cars the model looked at).
 - The MCTS tree with visit counts and Q-values.
 - Training curves (imitation loss, PPO reward).
 - Teacher vs Student vs Student+MCTS in Compare Mode.

Concepts for the viva: MCTS (AIMA §5.4), exploration vs exploitation (UCB/PUCT), chance nodes and expectimax.

Done when: the invariants hold at every tick, the time budget is met, and MCTS matches or beats RL alone on average wait (or this is honestly reported).

Phase 7 — Evaluation, docs and demo readiness (Days 6–7)

Prove the result with numbers, prepare every team member for questions, and merge.

Build

- **Final benchmark:** 5 strategies × 6 scenarios × 10 seeds; mean with 95% confidence intervals, a paired significance test, and wait-time distributions.
- **Ablations:** imitation only (no RL), no MCTS, no attention (plain MLP), no classical features.
- **Generalisation:** unseen 20-floor/6-car and 40-floor/8-car buildings, plus the breakdown and fire scenarios.
- **Docs:** `DESIGN.md` (LiftZero architecture, MDP, training, MCTS), `TESTING.md`, `VIVA_QA.md` (+20 questions), `DEMO_SCRIPT.md` (6 minutes using Story and Compare mode), and slide-ready charts.
- Merge `v2` → main, tag `v2.0`, and run the laptop rehearsal checklist.

Done when: all tests are green, the demo runs offline end to end, and every team member can present their phase.

Risks and fallbacks

The biggest risk is that RL does not beat the teacher. Even then, the demo holds up because every phase ends working.

Risk	Likelihood	Fallback
RL does not beat the teacher	Medium	Ship imitation + MCTS, report it honestly; the classical system still demos
Training too slow on 2 CPUs	Medium	Fast twin simulator, small model, curriculum, cap at 1M decisions
React UI takes longer than planned	Medium	Mission Control and Compare Mode first; the old UI stays at <code>/classic</code>
Twin simulator drifts from the Mesa model	Low-medium	Fidelity test; fine-tune on Mesa; always evaluate on Mesa
MCTS too slow live	Low	Top-3 cars, 64 simulations, budget cut-off falls back to the auction
Evaluation comes earlier than expected	Low	Every phase ends demoable; <code>v1-stable</code> tag to fall back on

Risk	Likelihood	Fallback
Laptop setup on demo day	Low	<code>uv sync</code> done in advance; ONNX models and built UI committed; no internet needed

Team ownership for the viva

Each member owns the phases they will explain. Everyone should be able to walk through the sense → reason → decide → act cycle and say why the system is multi-agent.

Member	Owns	Explains in the viva
A	Phase 1 — multi-agent core	Agent types, PEAS, environment, FIPA-ACL, Contract Net
B	Phases 3-4 — environment, dataset, Transformer	MDP formulation, features, attention, imitation learning
C	Phases 5-6 — PPO and MCTS	Reinforcement learning, cooperative learning, MCTS and search
D	Phases 2 and 7 — UI, benchmark, docs	Live demo, results, testing, tool choices

How we work, phase by phase

You start each phase; I build, test and commit it; you check it and approve before the next one.

1. You say "Start Phase N".
2. I build it on the `v2` branch in your project folder, run the tests, and commit.
3. I report what changed, the test results, and the exact commands to check it.
4. You run the commands, look at the app, and reply "ok" or list changes. Then we move to the next phase.

Notes

- Git has to delete its own lock files, so at the start of Phase 1 I will ask once for delete permission on the project folder, for git only.

- Training runs in my cloud workspace. Only the finished model files (under 5 MB) go into your repo.

Check commands after each phase

```
cd ~/adhikkesh/SEM7/FOAI/Multi-Agent-Smart-Elevator-Fleet-Coordinator
git checkout v2
uv sync
uv run pytest
uv run elevator verify
uv run elevator serve    # then open http://localhost:8000
```